

Radar de Erros da API

Análise de 1.000 requisições de API — onde os erros se concentram, o que eles custam em latência, e o que investigar primeiro.

Taxa de erro geral 28,6%

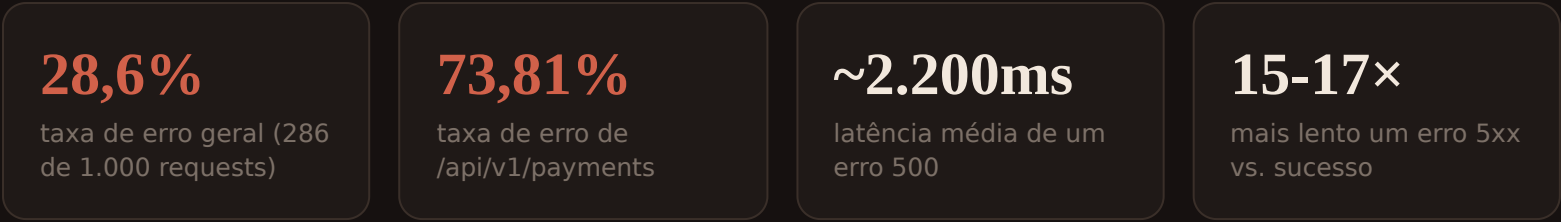
/api/v1/payments em 73,81%

Erros 5xx custam +2.000ms

Query de monitoramento incluída

FONTE	AMOSTRA	ENDPOINTS	GERADO EM
api_logs.xlsx	1.000 requests	10 rotas	25 set 2026

Um endpoint concentra a maior parte do problema

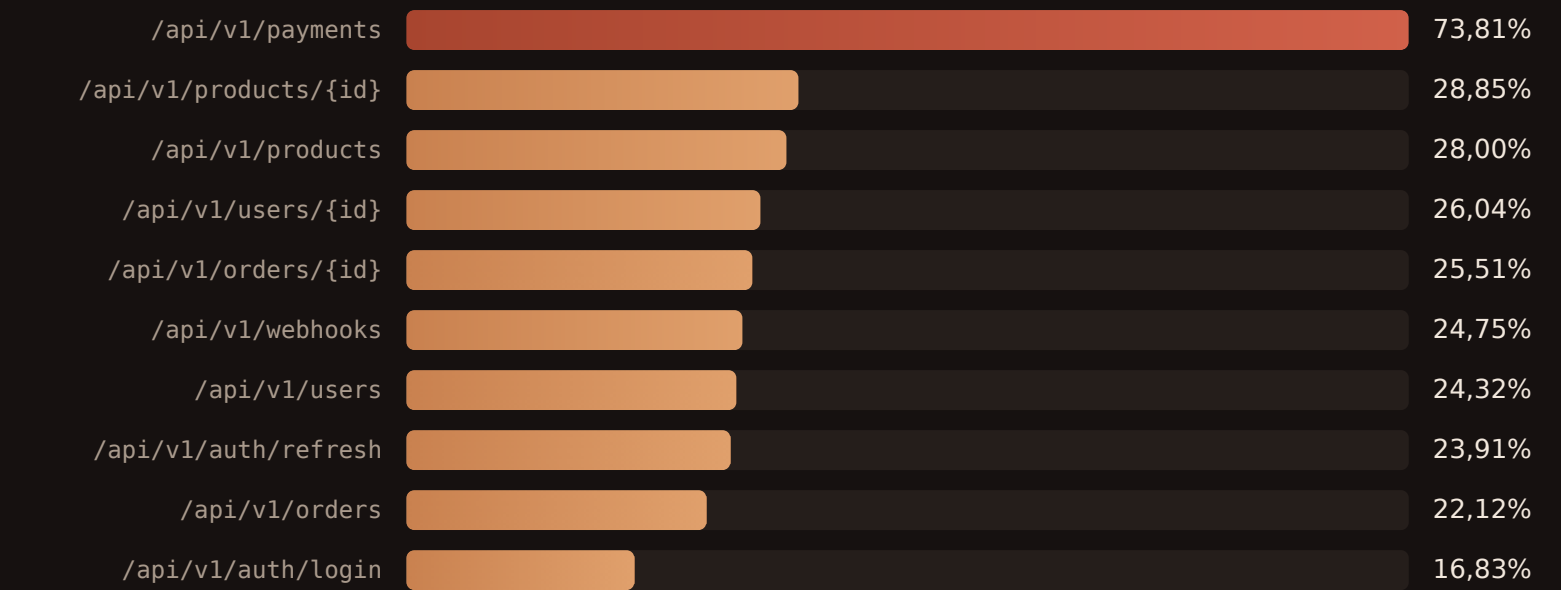


- **/api/v1/payments é o principal ponto crítico:** taxa de erro de 73,81%, quase 3× o segundo colocado, com latência consistentemente mais alta em qualquer status code.
- **Erros 5xx (500/502) são o gargalo de latência real:** requests que falham demoram em média 2.000-2.200ms — padrão típico de timeout em dependência downstream.
- **Não há pico horário dominante para erros 500:** distribuição relativamente uniforme ao longo do dia, com leve concentração de madrugada puxada por `/api/v1/products/{id}`.
- **Erro e latência caminham juntos:** cruzando as duas métricas por endpoint, payments é o único outlier isolado — os outros nove formam um cluster coerente.

Como os números foram calculados

- **Fonte:** planilha `api_logs.xlsx`, aba "API Logs", com 1.000 registros (timestamp, endpoint, método, status code, latência, tamanho de payload, serviço, região, usuário).
- **Período:** requests distribuídas a partir de 01/03/2024 — amostra, não uma série contínua de várias semanas.
- **Definição de erro:** status code ≥ 400 (4xx de cliente e 5xx de servidor). Onde relevante, 5xx é analisado separadamente por representar falhas de infraestrutura.
- **Ressalva estatística:** apenas 34 erros 500 na amostra — diferenças de 1-2 ocorrências por hora não configuram um padrão operacional forte.

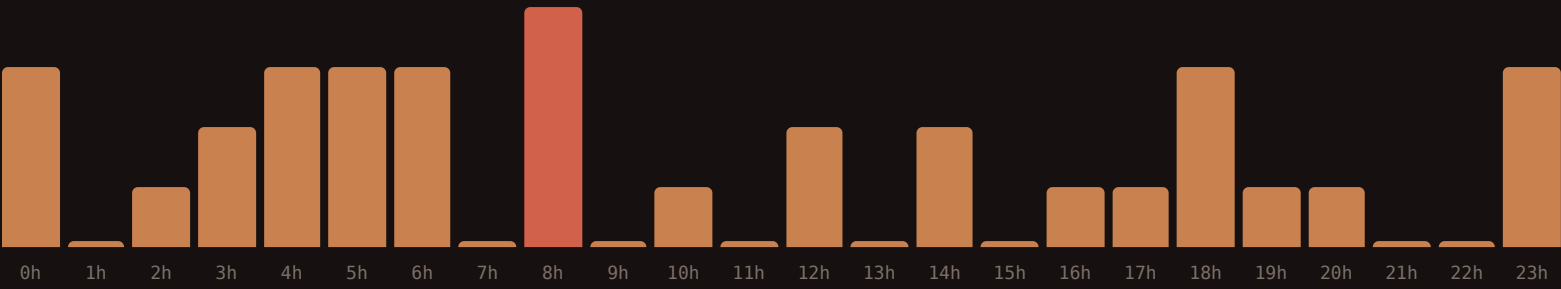
Taxa de erro por endpoint



`/api/v1/payments` é um outlier claro: quase 3× a taxa de erro do segundo colocado, com quase 3 em cada 4 requests falhando. Os principais status nesse endpoint são 400 (14), 404 (8), 500 (7), 502 (7), 503 (6) e 429 (5) — uma mistura de erros de cliente, servidor e indisponibilidade, sugerindo instabilidade generalizada, não só validação de entrada.

Erros 500 ao longo do dia

Dos 1.000 requests, 34 retornaram status 500. O pico isolado é 8h (4 ocorrências), mas com diferença mínima para vários outros horários — dado o baixo volume total, não é um padrão forte.



Cruzamento com endpoint

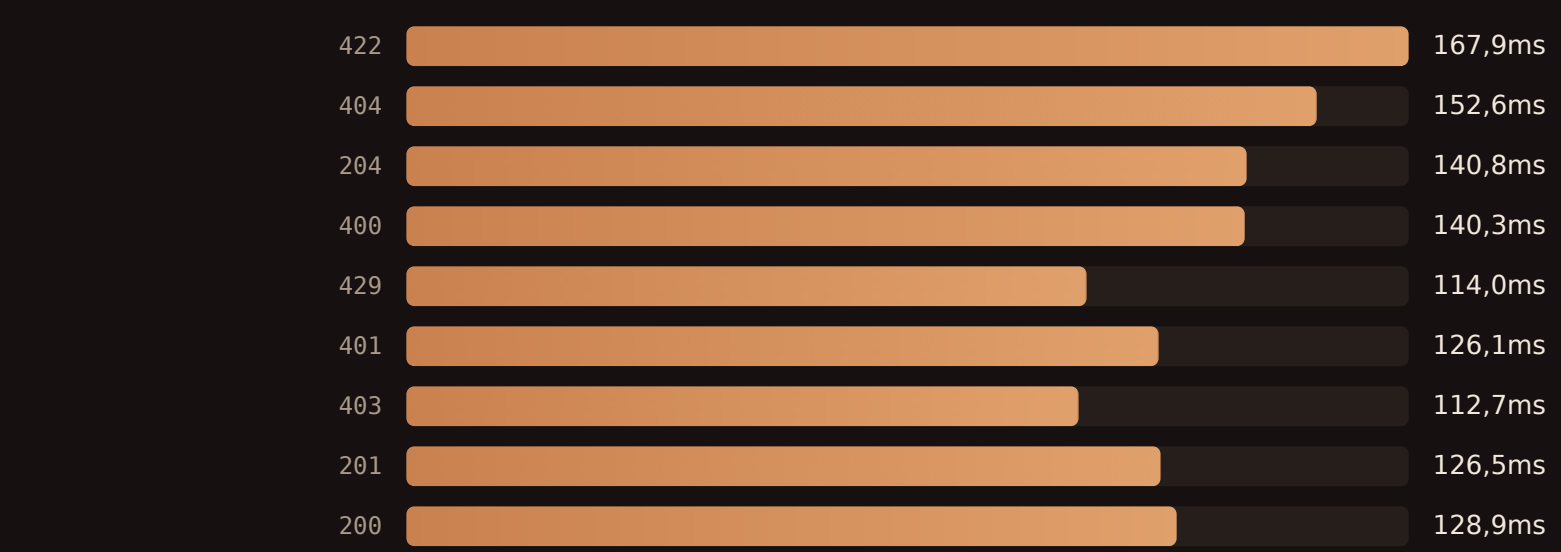
HORA	PAYMENTS	PRODUCTS	WEBHOOKS	PRODUCTS/{ID}
0h	-	-	1	1
3h	1	-	-	-
4h	-	1	-	1
5h	-	1	-	2
6h	1	-	-	-
8h	-	2	-	-
12h	1	-	-	-
18h	1	-	-	-
20h	1	-	-	-
23h	2	-	-	-

O pico de 8h é explicado inteiramente por `/api/v1/products`. A concentração de madrugada (0h-6h) é puxada majoritariamente por `/api/v1/products/{id}` — não por `/api/v1/payments`, que está disperso ao longo do dia. Com células de 1-2 ocorrências, isso ainda é mais ruído estatístico do que um padrão operacional confiável.

Latência média por endpoint e por status code

ENDPOINT	REQUESTS	MÉDIA (MS)	MEDIANA (MS)	P95 (MS)
/api/v1/payments	84	360,9	354,8	491,6
/api/v1/products	100	293,7	117,3	2.480,4
/api/v1/products/{id}	104	285,8	119,0	2.094,8
/api/v1/webhooks	101	245,8	117,3	208,4
/api/v1/users/{id}	96	242,9	105,9	619,6
/api/v1/orders/{id}	98	238,9	115,1	186,1
/api/v1/orders	113	236,5	117,3	219,8
/api/v1/auth/refresh	92	208,0	110,8	213,9
/api/v1/users	111	189,9	112,1	209,7
/api/v1/auth/login	101	132,8	114,2	204,3

Latência por status code — sucesso e erros de cliente

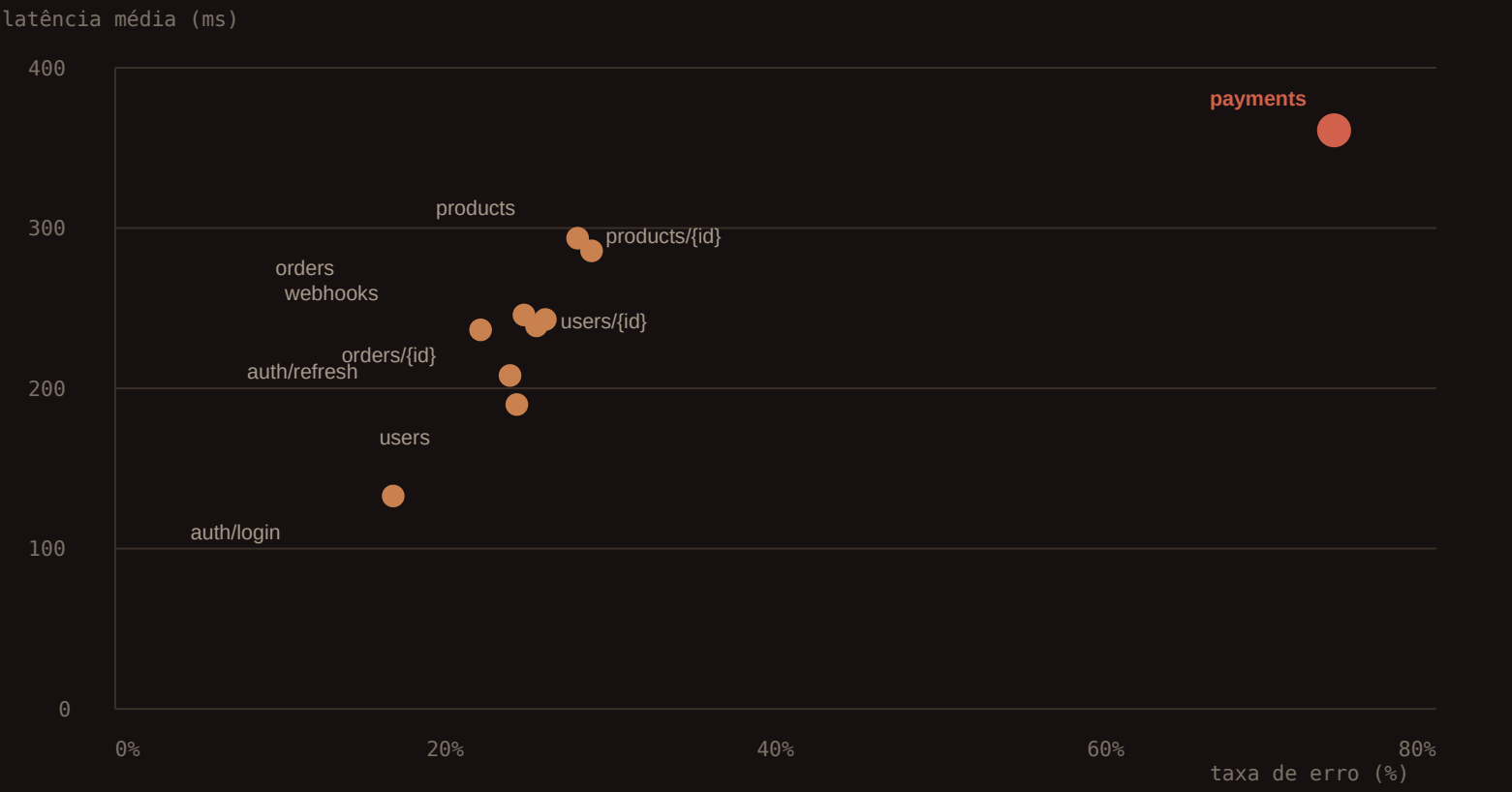


Latência por status code — erros de servidor (escala própria)



Requests com status 500 ou 502 demoram, em média, 15-17× mais que uma request bem-sucedida (~130ms). Note a escala própria do segundo gráfico — na mesma escala do primeiro, essas barras sairiam da página.

Onde erro e latência se cruzam



Nove dos dez endpoints formam um cluster coerente: taxa de erro entre 17-29%, latência entre 130-290ms — comportamento de API com espaço pra melhorar, mas numa faixa razoável. `/api/v1/payments` está isolado, sozinho, no canto superior direito.

Não é um efeito colateral, é outra categoria de problema. Payments falha quase 3 em cada 4 vezes e é o mais lento em qualquer cenário — diferente dos demais endpoints, cuja lentidão vem quase toda dos erros 5xx pontuais (timeout), payments é lento mesmo quando dá certo.

Os quatro achados que mais importam

- Erros 5xx são o gargalo de latência mais grave:** 500/502 levam ~2.000-2.200ms em média — timeout em dependência downstream (espera, tenta, falha, só então retorna o erro).
- products e products/{id} têm o pior p95** (2.480ms e 2.095ms) apesar de mediana baixa — distribuição bimodal: maioria rápida, uma cauda de segundos.
- payments tem latência estruturalmente alta e estável** (330-400ms independente do status) — sugere processamento síncrono mais pesado, não picos de timeout.

- **Piores casos pontuais:** orders/{id} com 500 (3.358ms), users/{id} com 502 (2.897ms) e webhooks com 500 (2.829ms) travam a request por quase 3 a 3,4 segundos.

— PLANO DE AÇÃO

Recomendações priorizadas

1 Investigar /api/v1/payments com urgência.

Taxa de erro de 73,81% é incompatível com um serviço em produção estável. A mistura de 400/404/500/502/503/429 sugere instabilidade generalizada, não um bug pontual.

2 Aplicar timeout agressivo (500ms-1s) nas chamadas downstream de products e products/{id}.

O ganho mais rápido para reduzir o p95 sem mexer na lógica de negócio: falhar rápido em vez de segurar a conexão por segundos.

3 Configurar alertas de monitoramento em tempo real por serviço e endpoint.

Usando a query SQL abaixo, com threshold de taxa de erro (ex.: >5%) e volume mínimo (ex.: ≥ 10 requests) para evitar falsos positivos.

4 Coletar mais dados antes de investir em análise horária.

Com apenas 34 erros 500 na amostra, qualquer padrão por hora do dia tem ruído estatístico relevante; acumule várias semanas antes de decisões operacionais baseadas em horário.

Query SQL — taxa de erro por serviço nas últimas 24h

PostgreSQL, pronta para rodar num dashboard ou alerta agendado.

```
SELECT
  service,
  COUNT(*) AS total_requests,
  COUNT(*) FILTER (WHERE status_code >= 400) AS total_errors,
  COUNT(*) FILTER (WHERE status_code >= 500) AS server_errors_5xx,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE status_code >= 400) / NULLIF(COUNT(*), 0),
    2
  ) AS error_rate_pct,
  ROUND(AVG(latency_ms) FILTER (WHERE status_code >= 400), 2) AS avg_latency_erros_ms,
  MAX(timestamp) AS last_seen
FROM api_logs
WHERE timestamp >= NOW() - INTERVAL '24 hours'
GROUP BY service
ORDER BY error_rate_pct DESC;
```

Quebra por endpoint (filtra baixo tráfego)

```
SELECT
  service,
  endpoint,
  COUNT(*) AS total_requests,
  COUNT(*) FILTER (WHERE status_code >= 400) AS total_errors,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE status_code >= 400) / NULLIF(COUNT(*), 0),
    2
  ) AS error_rate_pct
FROM api_logs
WHERE timestamp >= NOW() - INTERVAL '24 hours'
GROUP BY service, endpoint
HAVING COUNT(*) >= 10
ORDER BY error_rate_pct DESC
LIMIT 20;
```

Performance: índice composto `(timestamp, service)`; considerar particionamento diário para volumes altos; manter o filtro `HAVING COUNT(*) >= 10` em produção para evitar falsos positivos.